

CSE 232 : Data Structures

LAB 07 : BINARY SEARCH TREE

Due Saturday at 5:00 p.m. MST

The seventh programming assignment of this semester is to create a `BST` container. This is not a standard template library container. We will create `std::set` and `std::map` from the `BST`.

BST

Your `BST` class will be in `bst.h` and be named `custom::BST`. It will need to support the following operations:

- **Construct:** default constructor, copy constructor, move constructor, and initializer list constructor.
- **Assign:** copy assign, move assign, initializer list assign, and swap.
- **Iterator:** begin and end (for the 100 and 115 options only).
- **Access:** find.
- **Insert:** copy insert, move insert, initializer list insert, and range insert.
- **Remove:** erase and clear.
- **Status:** size and empty.

Three Options

This week, there will be three options.

85 Option : No Iterator or Red/Black

Unlike previous weeks, the 85 option does require a template parameter for the underlying data type.

```
template <typename T>
```

There are two member variables. The first is the binary node type from last week which will be a nested class.

```
class BNode;  
  
BNode* root;           // root node of the binary search tree  
size_t numElements;    // number of elements currently in the tree
```

There is no iterator nor red/black balancing here. The `find()` method will return a pointer to the element in the binary tree. There are two options: read-only and read/write. The `erase()` method will return a `bool`, indicating whether an element was erased from the `BST`. Note that if the element does not exist, then it will return `false`. To complete the 85 option, please start with the following files: [232.07.Lab.85.zip](https://content.byui.edu/file/6b80dd73-8931-4961-b7c5-f9660b343a18/1/Ponder/232.07.Lab.html).

100 Option : Iterator

The 100 option allows the client to specify the underlying data type through a template parameter.

```
template <typename T>
```

This option will have the same member variables as the 85 option. Additionally, we will be creating a BST iterator. Only a read/write iterator is needed, but this iterator will be bi-directional. The following methods will be supported:

- **Construct:** default constructor, copy constructor, non-default construct (so `begin()` and `end()` can properly initialize it).
- **Assign:** copy assign.
- **Compare:** equal-to and not-equal-to.
- **Access:** dereference (read-only).
- **Arithmetic:** increment (prefix and postfix), decrement (also prefix and postfix).

To complete the 100 option, please start with the following files: [232.07.Lab.100.zip](#).

115 Option : Red/Black Balancing

This option will require the class to everything from the 100 option. Additionally, it will be necessary to balance the BST using the Red/Black algorithm. It will allow the client to specify the underlying data type through a template parameter.

```
template <typename T>
```

This option will have the same member variables as the 85 and 100 options. To complete the 115 option, please start with the following files: [232.07.Lab.115.zip](#).

As with last week, the 115 option requires a precise number of allocations and deletions. In order to have more control over this, it is necessary to use an allocator. Please see Appendix I: Allocator in the text book for a detailed description of how this works. There is also a video demonstration here: [Allocators and Spys](#). This explains how the `spy` class works to count allocations, and how `std::allocator` can be use to more precisely control them.

Assignment

Please do the following:

1. Select an option: 85, 100, or 115.
2. Download the code.
3. Implement the data structure with your partner(s).
4. Report how long it took for each individual, and what was the hardest part.
5. Upload your `bst.h` code.

Assessment

The grade for your assignment will be based on the following criteria:

Exceptional 100%	Good 90%	Acceptable 70%	Developing 50%	Missing 0%
---------------------	-------------	-------------------	-------------------	---------------

Code Quality 35%	The most elegant solution was found for all the methods; there is no room for improvement,	There are no errors in efficiency, maintainability, cohesion, robustness, or redundancy.	There exists one error in efficiency, maintainability, cohesion, robustness, or redundancy.	The code quality is not what is expected for a CSE 232 student.	The code is sloppy or otherwise unprofessional.
Functionality 50%	Passes all the tests	Passes 90% of unit tests	Passes 70% of unit tests	Passes 50% of unit tests	Program fails to compile or was not submitted.
Iterator 15%	The 100 or the 115 solution was attempted				The 85% solution was attempted
Red/Black Balancing 15%	115 solution was attempted				The 85 or the 100 solution was attempted